DIGITAL IMAGE PROCESSING

LECTURE # 12

IMAGE COMPRESSION-II

Engr. M. Bilal

Topics to Cover

- Variable Length Coding
  - Huffman Coding
- Mapping
  - Difference Coding
  - Run-Length Coding
- LZW Coding
- Bit Plane Coding
  - Constant Area Coding
  - Quad Tree Coding

Variable-length Coding Methods: Huffman Coding

The most popular technique for removing coding redundancy; yields the smallest possible number of code symbols per source symbol

Huffman Coding Algorithm

- Arrange the symbol probabilities p_i in a decreasing order; consider them (p_i) as leaf nodes of a tree
- While there is more than one node:
  - *merge the two nodes with smallest probability to form a new node whose probability is the sum of the two merged nodes*
  - *Arrange the combined node according to its probability in the tree*
  - *Repeat until only two nodes are left*
- Starting from the top, arbitrarily assign 1 and 0 to each pair of branches merging into a node
- Continue sequentially from the root node to the leaf node where the symbol is located to complete the coding

Variable-length Coding Methods: Huffman Coding

Coding and decoding are done by simple look-up tables

Example:

Original source		Source reduction			
Symbol	Probability	1	2	3	4
a_2	0.4	0.4	0.4	0.4	0.6
a_6	0.3	0.3	0.3	0.3	0.4
a_1	0.1	0.1	0.2	0.3	
a_4	0.1	0.1	0.1		
a_3	0.06	0.1			
a_5	0.04				

FIGURE 8.11 Huffman source reductions.

Original source			Source reduction									
Sym.	Prob.	Code	1		2		3		4			
a_2	0.4	1	0.4	1	0.4	1	0.4	1	0.6	0		
a_6	0.3	00	0.3	00	0.3	00	0.3	00	0.4	1		
a_1	0.1	011	0.1	011	0.2	010	0.3	01				
a_4	0.1	0100	0.1	0100	0.1	011						
a_3	0.06	01010	0.1	0101								
a_5	0.04	01011										

FIGURE 8.12 Huffman code assignment procedure.

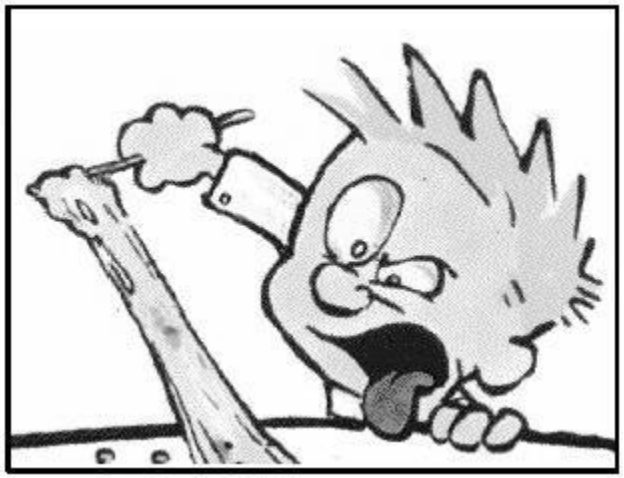
Variable-length Coding Methods: Huffman Coding

The Huffman code (continued)

- The Huffman code results in an unambiguous code, i.e. no code can be created by combining other codes.
- The code is reversible without loss.
- The table for the translation of the code has to be stored together with the coded image.

Mapping

- There is often correlation between adjacent pixels, i.e. the value of the neighbors of an observed pixel can often be predicted from the value of the observed pixel. (Interpixel Redundancy).
- Mapping is used to remove Interpixel Redundancy.
- Two mapping techniques are:
  - ✓ **Run length coding**
  - ✓ **Difference coding.**



Run-Length Coding

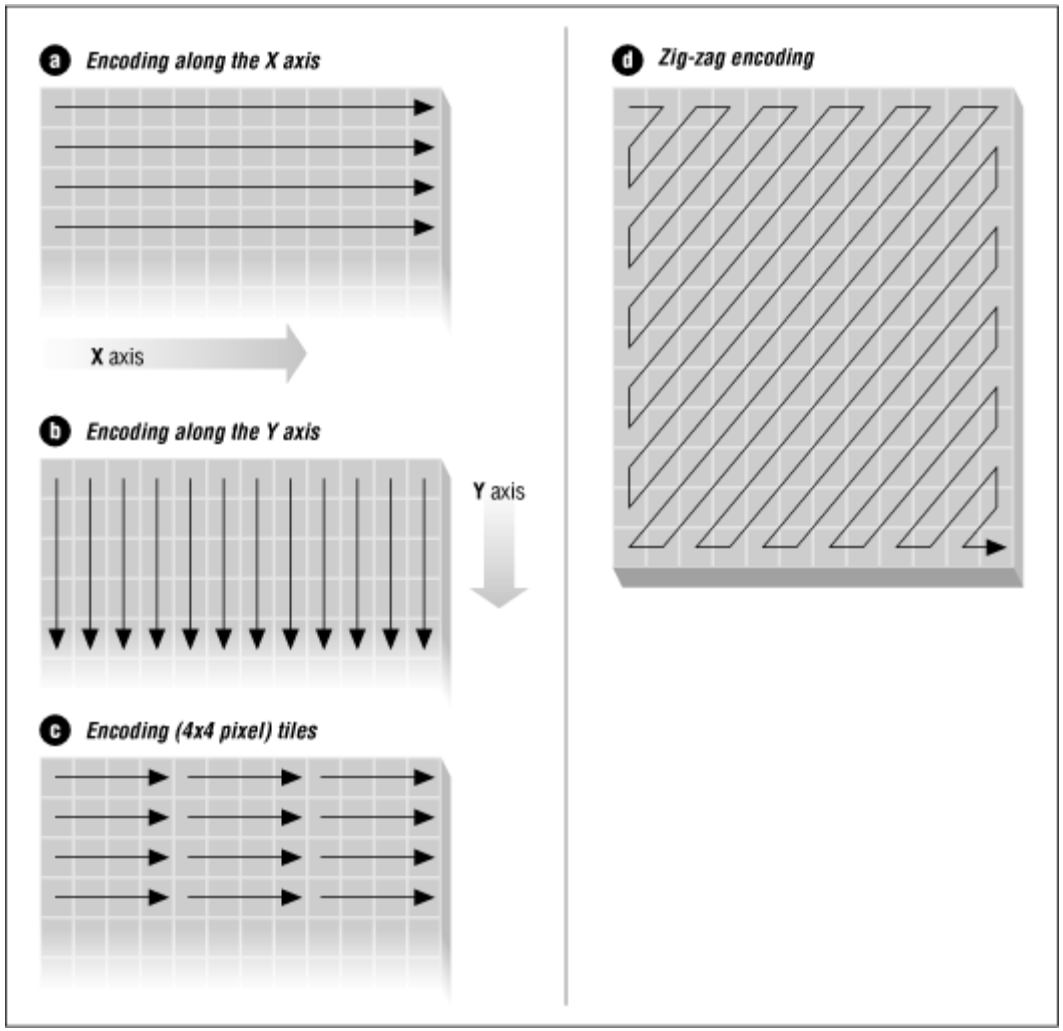
Every code word is made upp of a pair **(g,l)** where **g** is the graylevel and **l** is the number of pixels with that graylevel (length, or "run").

Ex 56 56 56 82 82 82 83 80
56 56 56 56 56 80 80 80

creates the runlength code (56,3) (82,3) (83,1) (80,4) (56,5)

-The code is calculated row by row.
-Very efficient coding for binary data.
-Important to know position, and the image dimensions must be stored with the coded image.
-Used in most fax machines

Interpixel Redundancy

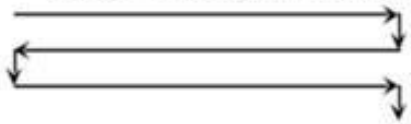


Difference Coding

$$f(x_i) = \begin{cases} x_i \text{ if } i=0, \\ x_i - x_{i-1} \text{ if } i>0 \end{cases}$$

Ex original:	56 56 56 82 82 82 83 80 80 80 80
code $f(x_i)$:	56 0 0 26 0 0 1 -3 0 0 0

- The code is calculated row by row.

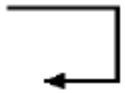


- Both Run-Length and Difference Coding are reversible and can be combined with other coding schemes such as Huffman Coding.

Example of Combined Difference and Huffman Coding

original image

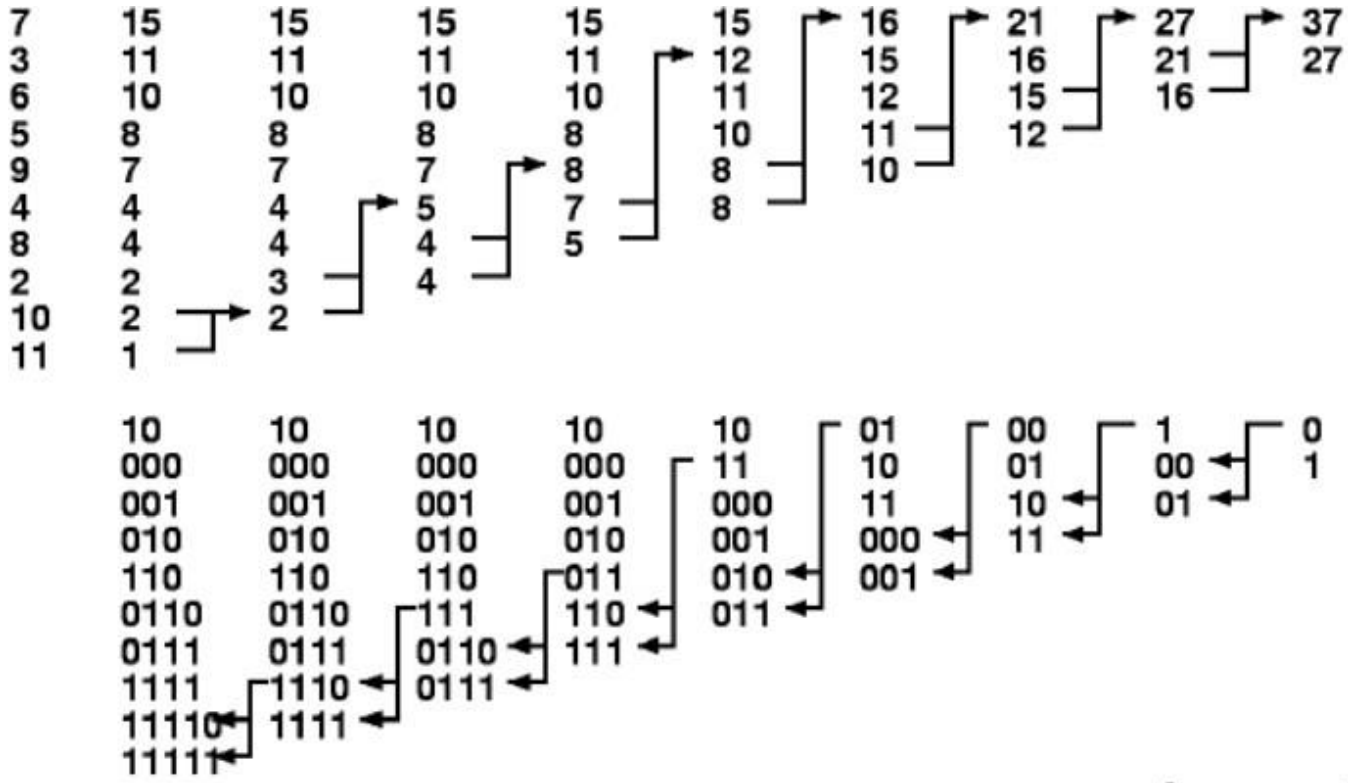
9	8	7	7	7	5	5	5
7	7	7	7	4	4	5	5
6	6	6	9	9	9	6	6
6	6	7	7	7	9	9	9
3	7	7	8	8	8	3	3
3	3	3	3	3	3	3	3
10	10	11	7	7	7	6	6
4	4	5	5	5	2	2	6



difference image

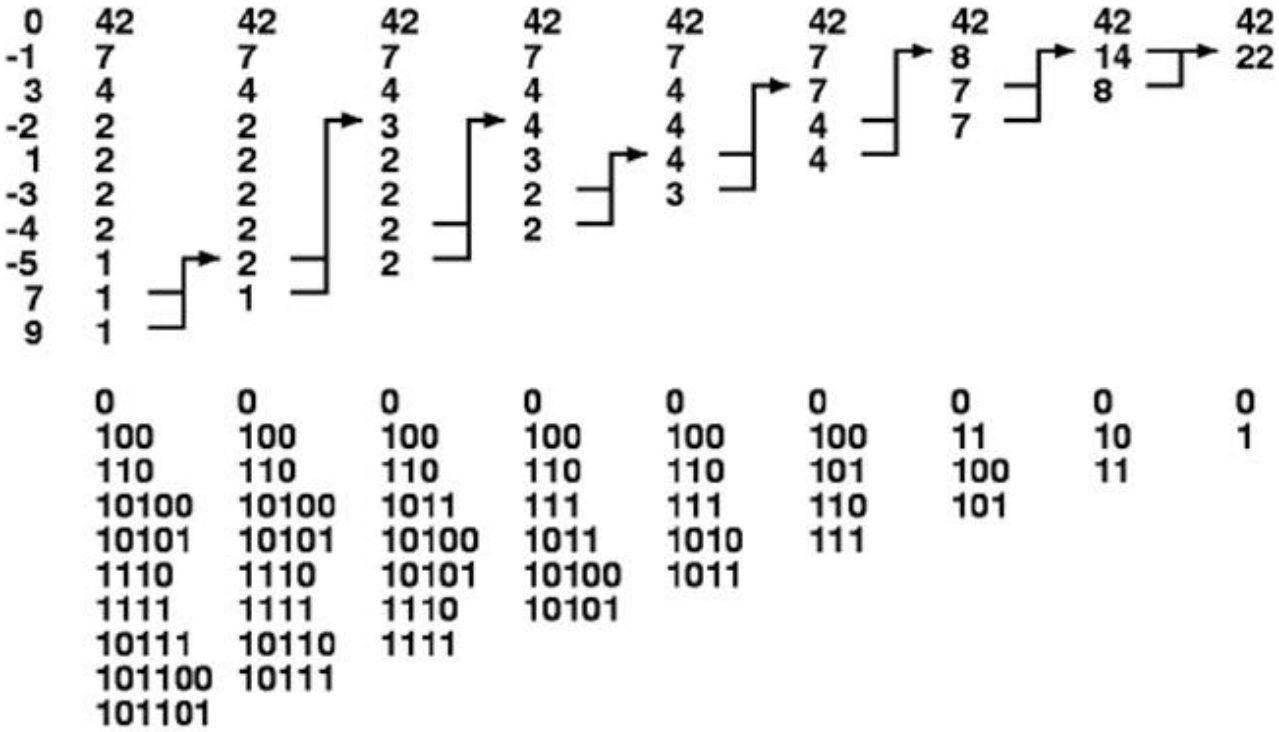
9	-1	-1	0	0	-2	0	0
0	0	0	3	0	-1	0	0
-1	0	0	3	0	0	-3	0
0	-1	0	0	-2	0	0	3
-3	4	0	1	0	0	-5	0
0	0	0	0	0	0	0	0
7	0	1	-4	0	0	-1	0
0	-1	0	0	3	0	-4	0

Huffman Code of Original Image



$L_{avg}=3.1$

Huffman Code of Difference Image



$L_{avg}=2$

LZW Coding (Dictionary based coding)

- Dictionary-based coding techniques are based on the idea of incrementally building a dictionary (table) while receiving the data. Unlike VLC techniques, dictionary-based techniques use fixed-length code words to represent variable-length strings of symbols that commonly occur together.
- Lempel-Ziv-Welch (LZW) coding assigns fixed length code words to variable length sequences of source symbols.
- It also takes care of the interpixel redundancies.
- Popular coding scheme used in formats like GIF, TIFF e.t.c

LZW Coding (Dictionary based coding)

- **METHOD**
  - A code book or dictionary containing the source symbols to be coded is constructed. For 8 bit monochrome images, the first 256 words of the dictionary are assigned to the gray values 0,1,2,…,255.
  - As the encoder sequentially examines the image's pixels, gray level sequences that are not in the dictionary are placed algorithmically determined (e.g., the next unused) locations.
  - If the first two pixels of the image are white, for instance, sequence "255- 255" might be assigned to location 256. the next time that two consecutive white pixels are encountered, code word 256, the address of the location containing sequence 255-255, is used to represent them.
  - A unique feature of LZW coding is that coding dictionary or code book is created while the data are being encoded.

LZW Coding (Dictionary based coding)

Consider the following 4 × 4, 8-bit image of a vertical edge:

39	39	126	126
39	39	126	126
39	39	126	126
39	39	126	126

Table 8.7 details the steps involved in coding its 16 pixels. A 512-word dictionary with the following starting content is assumed:

Dictionary Location	Entry
0	0
1	1
:	.
255	255
256	—
.	⋮
511	—

Locations 256 through 511 are initially unused.

LZW Coding (Dictionary based coding)

LZW coding for 2-D images

- Encoding is done by scanning left to right and top to bottom

Example: suppose a 4x4 8-bit image →

39	39	126	126
39	39	126	126
39	39	126	126
39	39	126	126

Currently Recognized Sequence	Pixel Being Processed	Encoded Output	Dictionary Location (Code Word)	Dictionary Entry
	39			
39	39	39	256	39-39
39	126	39	257	39-126
126	126	126	258	126-126
126	39	126	259	126-39
39	39			
39-39	126	256	260	39-39-126
126	126			
126-126	39	258	261	126-126-39
39	39			
39-39	126			
39-39-126	126	260	262	39-39-126-126
126	39			
126-39	39	259	263	126-39-39
39	126			
39-126	126	257	264	39-126-126
126		126		

Error Free Coding

- **Bit-plane Coding**

✓ we can decompose a m-bit image into m bit planes (or gray-code planes)

✓ Each such plane is a binary image which can be coded in many ways, choose methods most appropriate for each particular bit plane

✓ Focus on reducing inter-pixel redundancy

✓ First decompose the original image into bit-plane and then apply binary image compression approach “run-length coding (RLC)”

Constant Area Coding

- Divide each bitplane (or binary image) into non-overlapping regions of m × n pixels
- Each such block is either all white, all black, or mixed
- Assign 1-bit codeword 0 to the block type that occurs most often
- Assign 10 and 11 to other two block types
- For the mixed-block, the assigned codeword can only serve as a *prefix*, the mn-bit pattern of the block must be appended after the prefix code-word
- Useful when many blocks are all white or all black

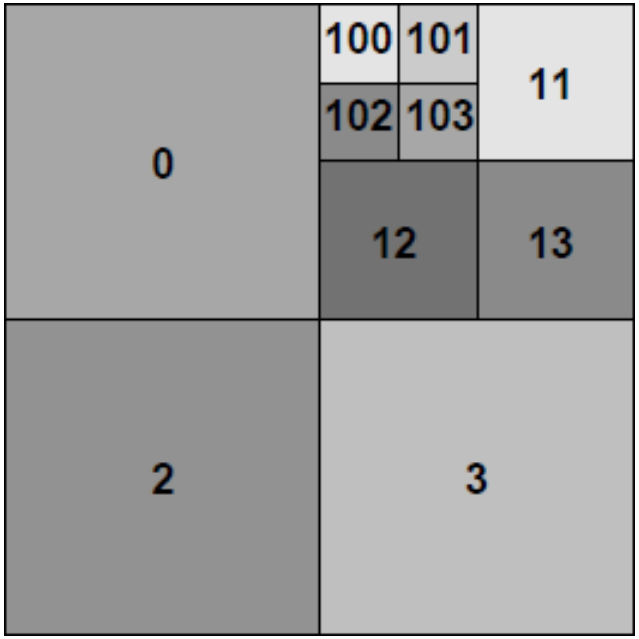
Quad Tree Codes

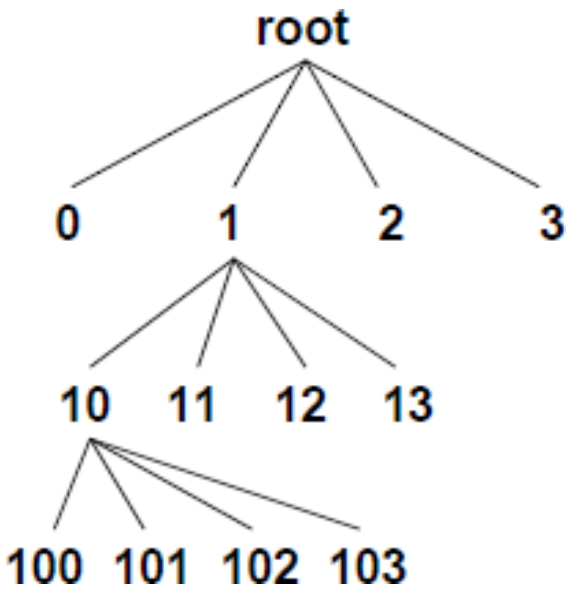
- Divide each binary image into 4 quadrants, each of which' is a block of type — all white, all black, or mixed

1	2
3	4

- Select a coding order for the 4 quadrants e.g.
- Assign unique code to all four quadrants, e.g. 00, 01, 10 and 11
- Repeat the quadrant-coding procedure for mixed blocks, and append the corresponding quadrant codes after the prefix
- For binary images with strong interpixel redundancy, we can expect quad-tree codes to give good compression

Quad Tree Structure





References

1. DIP by Gonzalez

For any query Feel Free to ask